

CSCE 605 COMPILER DESIGN · MODULE 08 · LECTURE NOTES

Dataflow Analysis

Facts that hold at every program point, computed by iteration.

MODULE OUTCOMES

Formulate an analysis as a lattice plus transfer functions.

Explain why the iteration terminates and what it converges to.

Implement liveness, reaching definitions, and available expressions.

Distinguish forward from backward and may from must.

Explain what static analysis fundamentally cannot do.

1 The dataflow framework

EVERY DATAFLOW ANALYSIS IS THE SAME FOUR THINGS

A lattice of facts. The set of possible answers, equipped with a partial order (how precise one answer is relative to another) and a meet operation for combining answers arriving from different paths.

A transfer function per instruction, describing how that instruction transforms the facts holding before it into the facts holding after it.

A direction. *Forward* analyses propagate facts along control flow (what has happened already); *backward* analyses propagate against it (what will happen next).

A meet operator. **Intersection for *must* analyses** (the fact holds on *all* paths) and **union for *may* analyses** (the fact holds on *some* path). **Choose those four and the algorithm is already written** — which is why this is called a framework: the iteration below is shared by every analysis in §2.

```

initialise every block's OUT to lattice TOP
worklist = all blocks

while worklist not empty:
    b = worklist.pop()
    IN[b] = MEET over predecessors p of OUT[p]      # forward
    new = TRANSFER(b, IN[b])
    if new != OUT[b]:
        OUT[b] = new
        worklist += successors(b)

TERMINATES because: the lattice has FINITE HEIGHT, and transfer
functions are MONOTONE -- more input facts never yield fewer
output facts -- so each OUT only moves DOWN, finitely often.

CONVERGES TO: the maximal fixpoint. The most precise answer this
method can give -- not necessarily the TRUE answer (section 4).

```

Worklist order affects speed, never the result. Reverse postorder for forward analyses and postorder for backward ones typically converge several times faster than arbitrary order, because they tend to process a block after its predecessors have settled. **That the final answer is order-independent is the fixpoint property**, and it is worth asserting in tests: shuffling the worklist and getting a different answer means a non-monotone transfer function, which is a bug.

2 The classical analyses

Analysis	Direction and meet	What it enables
Liveness	Backward, union (may).	Register allocation (Module 11) and dead store elimination. See the callout below.
Reaching definitions	Forward, union (may).	Use-def chains, and hence constant propagation. Made unnecessary by SSA (§4).
Available expressions	Forward, intersection (must).	Common subexpression elimination — an expression already computed on <i>every</i> path reaching here need not be recomputed.
Very busy expressions	Backward, intersection (must).	Code hoisting, and hence code size reduction — an expression computed on every path <i>from</i> here can be computed once, now.
Constant propagation	Forward, with a special three-level lattice (unknown / constant c / not constant).	Constant folding and branch elimination, which together frequently unlock everything else.
Alias analysis	Forward, may.	Everything that touches memory — and it is the bottleneck (§3).

The may/must and union/intersection pairing is the thing to memorise, and getting it backwards is not a precision bug but a soundness bug: a *may* analysis that meets with intersection under-approximates,

which licenses transformations that are not valid on all paths.

LIVENESS IS THE ONE YOU CANNOT AVOID

A variable is live at a program point if its current value may be read at some point in the future before being overwritten. Note 'may' — liveness along any single successor path is enough.

It is backward because liveness flows from uses back toward definitions, and the meet is union because a value live on *any* successor path must be kept.

Register allocation is built directly on it: two values can share a register exactly when they are never simultaneously live, which is what the interference graph of Module 11 encodes. **Without liveness there is no register allocation,** which is why this is the analysis every compiler has.

And it finds dead stores for free: a definition whose variable is not live immediately after it can never be read, so the definition is dead and can be deleted — which in turn may make its operands dead, so the elimination iterates.

3 Precision, and what it costs

Axis	Cheap version	Precise version
Flow sensitivity	Flow-insensitive: one fact per variable for the whole function. Very fast.	Flow-sensitive: a fact per variable per program point. The framework above. Linear in program points, which is affordable.
Path sensitivity	Path-insensitive: merge facts at every join, losing the correlation between branches.	Path-sensitive: track facts per path. Exponential in the number of branches, so used only in bug-finding tools with aggressive path pruning.
Context sensitivity	One summary per function, merging all call sites.	A separate analysis per call site, or per call <i>string</i> — exponential in call depth. k-limiting is the usual compromise.
Field sensitivity	Treat an object as one indivisible thing.	Track each field separately. Affordable and usually worth it.
Heap modelling	One abstract node per allocation site, so all objects from one new are conflated.	Shape analysis, distinguishing individual heap cells. Very expensive, and mostly confined to verification tools.

Every one of these axes trades precision against cost exponentially, which is why production compilers sit near the cheap end on most of them and accept conservative answers. 'Just be more precise' is not an available option, and a compiler that spent exponential time to find one more constant would not be used.

ALIAS ANALYSIS IS WHAT LIMITS EVERYTHING ELSE

The question is: can these two pointers refer to the same storage? The possible answers are no, yes, and maybe — and if the answer is 'maybe', almost every memory optimisation is blocked.

And the answer is usually 'maybe', because proving two pointers distinct generally requires reasoning about the whole program, including code the compiler cannot see.

So a store through an unknown pointer invalidates every cached load, a value cannot be held in a register across it, loops cannot be vectorised across it, and instructions cannot be reordered around it. The conservative assumption is correct and extremely expensive.

This is precisely why shader languages forbid pointers (Module 06 §4), why C has restrict, why Fortran's stricter aliasing rules let it beat C on numerical code for decades, and why Rust's borrow checker lets rustc emit noalias on essentially every reference. **Four apparently unrelated language features that are all the same answer to this one analysis problem** — which is a good illustration of how much language design is downstream of what the compiler can prove.

4 Limits

EVERY STATIC ANALYSIS IS CONSERVATIVE BY NECESSITY

Any non-trivial semantic property of programs is undecidable — Rice's theorem again (Module 06 §1). No analysis can be exactly right about whether a variable is live, whether an expression is constant, or whether two pointers alias.

So the analysis must err in a *safe* direction — reporting that something *may* happen when in fact it cannot, never that it cannot when in fact it may.

Which direction is safe depends entirely on how the result is used. For dead code elimination, over-estimating liveness is safe (you keep code you could have deleted) and under-estimating is catastrophic (you delete live code). For a different client the safe direction may be the opposite.

Getting the direction wrong produces a miscompile (Module 01 §4), not a missed optimisation — and it will be found by a user, months later, in release builds only. **This is the single thing to be most careful about in this module**, and it should be stated explicitly in the comment above every transfer function you write.

SSA MAKES SEVERAL OF THESE UNNECESSARY

Reaching definitions is the question SSA answers by construction (Module 07 §2). Each use already names its unique definition, so the analysis simply is not needed — the representation has the answer.

Constant propagation becomes local, or better, *sparse conditional* constant propagation walking the SSA graph — which is both faster than the dataflow version and **strictly more precise**, because it can prove a branch never taken and ignore the values flowing along it.

Liveness is still required, and is cheaper to compute in SSA because def-use chains are explicit and live ranges have a known structure.

And memory analyses are entirely unaffected. SSA applies to register-like values, not to the heap — memory still has aliasing, still has unknown stores, and still blocks everything. **Which is why alias analysis remains the bottleneck no matter how good the IR is**, and why the language-level answers in §3 matter more than any compiler improvement.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Cooper & Torczon — Engineering a Compiler, chapter 9	The framework of §1 and the analyses of §2 , with the termination argument done properly.
Kildall; and Nielson, Nielson & Hankin — Principles of Program Analysis	The lattice-theoretic foundations. Heavy going; useful for the monotonicity and fixpoint results.

Resource	What it covers for this module
Hind — Pointer Analysis: Haven't We Solved This Problem Yet? (free)	The §3 bottleneck, surveyed. The title is the conclusion.
Wegman & Zadeck — Constant Propagation with Conditional Branches (free)	Sparse conditional constant propagation — the §4 result that SSA makes an analysis both faster and more precise.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement the generic worklist algorithm, parameterised on direction, meet, and transfer.
- 2 Instantiate it for liveness and verify against hand-computed answers on a small CFG.
- 3 **Shuffle the worklist order and confirm the answer is unchanged**, then confirm the iteration count changes.
- 4 Implement reaching definitions and available expressions.
- 5 **Write a non-monotone transfer function deliberately** and observe the iteration fail to converge or converge inconsistently.
- 6 Implement dead store elimination from liveness and measure how many stores it removes on real code.
- 7 Implement a flow-insensitive and a flow-sensitive version of one analysis. Compare precision and runtime.
- 8 **Implement a simple alias analysis** (address-taken, then allocation-site-based) and measure how many loads each lets you cache.
- 9 Add restrict to a C function and compare the generated code with and without.
- 10 **Implement sparse conditional constant propagation** over SSA and find a case where it beats the dataflow version.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name the four components of a dataflow analysis.
- 2 Why does the iteration terminate, and what does it converge to?
- 3 Does worklist order affect the answer? What does it affect?
- 4 Give six analyses with their direction and meet.
- 5 Why do may analyses use union and must analyses intersection?

- 6 Why is liveness unavoidable?
- 7 Name five precision axes and the cost of each.
- 8 Why is alias analysis the bottleneck, and name four language-level responses.
- 9 Why must every analysis be conservative, and what happens if you err the wrong way?